

PHP / MySQL / React Developer

Job Description & Practical Assessment

Part 1 — Job Description

Role Summary

We are hiring a PHP / MySQL / React Developer with 3–5 years of hands-on experience to build and maintain full-stack web applications. Candidates must be strong in the mandatory core stack; experience with the optional technologies below is a plus.

Experience

- 3–5 years of professional software development experience (mandatory).

Mandatory Skills

- PHP — object-oriented PHP, Composer, PSR standards, REST API development
- MySQL — schema design, query writing, indexing, and optimization
- React — hooks, component architecture, state management, API integration

Good-to-Have / Optional Skills

- Node.js and PostgreSQL
- Docker — containerization of full-stack applications, Docker Compose
- Cloud platforms — AWS or GCP (deployment, storage, managed databases, CI/CD)
- AI agent tooling — Claude, Gemini, or similar (API integration, agentic workflows, prompt engineering)

Key Responsibilities

- Design, build, and maintain backend services in PHP with MySQL as the primary data store.
- Develop responsive, componentized front-end interfaces in React.
- Implement secure authentication and role-based authorization across the stack.
- Write clean, secure, well-tested code and participate in code reviews.
- Optimize database queries and application performance.
- Collaborate with product/design to translate requirements into technical solutions.
- Where applicable, containerize services with Docker and contribute to cloud deployment and AI-assisted development tooling.

Requirements

- 3–5 years building production web applications with PHP, MySQL, and React.
 - Solid understanding of relational database design and SQL performance tuning.
 - Practical experience implementing authentication and authorization (sessions/JWT, role-based access control).
 - Familiarity with Git, REST APIs, and standard software engineering practices.
 - Good communication skills and ability to work independently.
 - Exposure to Docker, Node.js/PostgreSQL, AWS/GCP, or AI agent tools (Claude, Gemini) is a strong plus, not mandatory.
-

Part 2 — Practical Assessment

Purpose & Format

This is a hands-on, project-based assessment — no written Q&A. Candidates complete a set of practical tasks and submit working code; technical discussion, code walkthrough, and follow-up questions happen face-to-face in the interview.

Tasks 1–3 are core and mandatory. Tasks 4–7 are optional bonus work — attempt only what's realistic in the time available; partial or described-but-not-built solutions are still acceptable if explained clearly in the README.

Task 1 — Full-Stack CRUD App: Task Manager (Core)

Build a small “Task Manager” feature end to end:

- Backend: PHP REST API to create, list, update, and delete tasks, backed by MySQL.
- Task fields: title, description, status (todo/in-progress/done), priority, due date, owner (user_id).
- Support filtering by status and basic pagination on the list endpoint.
- Frontend: React app that lists tasks, adds/edits a task, updates status, and deletes a task, with loading and error states.
- Clean, validated input on both client and server; sensible REST status codes.

Task 2 — Authentication & Authorization (Core)

Add secure auth and role-based access control on top of Task 1:

- User registration and login with securely hashed passwords (e.g., PHP password_hash/password_verify).
- Session-based or JWT-based authentication protecting all task API endpoints (401 for unauthenticated requests).
- Two roles at minimum — “admin” and “user”: a user can only see/manage their own tasks; an admin can see/manage all users' tasks (403 for unauthorized access).
- Protected routes on the React side — unauthenticated users are redirected to login; admin-only UI/actions are hidden from regular users.
- Logout, and at least a basic approach to token/session expiry.
- Bonus within this task: refresh tokens, password reset flow, or audit logging of admin actions.

Task 3 — Database Design & Query Optimization (Core)

Formalize and defend the schema behind Tasks 1–2:

- Provide a schema (DDL or diagram) for users, roles, and tasks with correct relationships, constraints, and indexes.
- Write 2–3 optimized queries the app actually uses — e.g., a user's tasks by status, an admin's view of all tasks with owner info, task counts per user.
- Include EXPLAIN output (or a written note) justifying your indexing choices and any query tuning done.

Task 4 — Containerization with Docker (Non Mandatory)

Package the application so it can be run with a single command:

- Write Dockerfiles for the PHP/MySQL backend and the React frontend.
- Provide a docker-compose.yml that wires up the app, database, and any other services (e.g., phpMyAdmin) together.
- Use environment variables / .env for configuration and secrets — nothing hardcoded in the image.
- App should come up cleanly with docker-compose up and be reachable locally without manual setup steps.

Task 5 — Node.js / PostgreSQL Variant (Non Mandatory)

Re-implement one endpoint from Task 1 or 2 (e.g., authenticated task listing) using Node.js/Express and PostgreSQL instead of PHP/MySQL, or clearly document what would need to change to port the backend. Highlight any meaningful differences in approach (e.g., query syntax, connection pooling, auth middleware).

Task 6 — Cloud Deployment: AWS / GCP (Non Mandatory)

Deploy the application (or a meaningful subset) to AWS or GCP, using free-tier services where possible. Include a short note on: services used (e.g., EC2/RDS/S3 or Compute Engine/Cloud SQL/Cloud Storage), how secrets/environment variables are managed, and whether any CI/CD is wired up. Deploying the Dockerized version from Task 4 is a plus.

Task 7 — AI Agent Feature: Claude / Gemini (Non Mandatory)

Add one AI-powered feature to the app using the Claude or Gemini API — for example, auto-suggesting a task's category/priority from its title and description, or a simple natural-language “create a task by describing it” flow. Include the prompt(s) used and how the response is parsed and applied in the app.

Submission Guidelines

- Git repository link (or zip) with a clear commit history.
- README covering: setup/run instructions (docker-compose preferred if Task 4 is attempted), which tasks were completed, which bonus tasks were attempted or only described, and time spent.
- Face-to-face round: candidate walks through the code, explains key decisions (especially around auth/authz and schema design), and answers follow-up technical questions live.